UNIVERSITÉ PARIS SACLAY
Master E3A/SETI T1

EXAMEN

T1 Architecture avancée
décembre 2016

Durée : 3h

Documents autorisés pour la deuxième partie de l'examen
Les différentes parties sont strictement indépendantes.

1 Prédiction de branchement [8 pt]

Soit le programme C suivant :

```
int array[1000] = { /* valeurs quelconques */ };
int sum1 = 0, sum2 = 0, sum3 = 0, sum4 = 0;
for (i = 0; i < 1000; i ++) // BR0 : branchement de boucle
{
  if (i % 4 == 0)      // BR1 : IF CONDITION 1
    sum1 += array[i]; // Pris
  else
    sum2 += array[i]; // Non pris

  if (i % 2 == 0)      // BR2 : IF CONDITION 2
    sum3 += array[i]; // Pris
  else
    sum4 += array[i]; // Non pris
}
```

On rappelle qu'en C, `x % y` renvoie le reste de la division entière de `x` par `y`.

Le but de l'exercice est de déterminer la qualité de la prédiction (% de prédictions correctes) pour les branchements BR0 (pris dans la boucle et non pris en sortie de boucle), et les branchements BR1 et BR2 à l'intérieur de la boucle, pour différents types de prédicteurs

1.1 Quelle est la qualité de la prédiction de chacun des branchements BR0, BR1 et BR2 pour un prédicteur 1 bit (pris/non pris ou P/NP). On supposera que tous les prédicteurs sont initialisés à "non pris" (NP).

Rep :

BR0 prédiction	*N*	*P*	*P*	*P*	*P*	*P*	*P*	*P*	*P*	*P*	...	*P*	*P*
BR0 comportement	**P**	*P*	*P*	*P*	*P*	*P*	*P*	*P*	*P*	*P*	...	*P*	**N**

2 erreurs en tout, 2/1000 erreur/itération

BR1 prédiction	*N*	*P*	*N*	*N*	*N*	*P*	*N*	*N*	*N*	*P*	...	*N*	*N*
BR1 comportement	**P**	**N**	*N*	*N*	**P**	**N**	*N*	*N*	**P**	**N**	...	*N*	*N*

0.5 erreur/itération

BR1 prédiction	*N*	*P*	*N*	*P*	*N*	*P*	*N*	*P*	*N*	*P*	...	*N*	*P*
BR1 comportement	**P**	**N**	**P**	**N**	**P**	**N**	**P**	**N**	**P**	**N**	...	**P**	**N**

1 erreur/itération

1.2 Quelle est la qualité de prédiction de chacun des branchements BR0, BR1 et BR2 pour un prédicteur 2 bits (fortement pris/faiblement pris/faiblement non pris, fortement non pris). On supposera que tous les prédicteurs sont initialisés à “fortement non pris”.

Rep :

BR0 prédiction	*N*	*n*	*p*	*P*	*P*	*P*	*P*	*P*	*P*	*P*	...	*P*	*P*
BR0 comportement	**P**	**P**	*P*	*P*	*P*	*P*	*P*	*P*	*P*	*P*	...	*P*	**N**

3 erreurs en tout, 3/1000 erreur/itération

BR1 prédiction	*N*	*n*	*N*	*N*	*N*	*n*	*N*	*N*	*N*	*n*	...	*N*	*N*
BR1 comportement	**P**	*N*	*N*	*N*	**P**	*N*	*N*	*N*	**P**	*N*	...	*N*	*N*

0.25 erreur/itération

BR1 prédiction	*N*	*n*	*N*	*n*	*N*	*n*	*N*	*n*	*N*	*n*	...	*N*	*n*
BR1 comportement	**P**	*N*	**P**	*N*	**P**	*N*	**P**	*N*	**P**	*N*	...	**P**	*N*

0,5 erreur/itération

1.3 Même question quand les prédicteurs sont initialisés à “faiblement non pris”.

Rep :

BR0 prédiction	*n*	*P*	*P*	*P*	*P*	*P*	*P*	*P*	*P*	*P*	...	*P*	*P*
BR0 comportement	**P**	*P*	*P*	*P*	*P*	*P*	*P*	*P*	*P*	*P*	...	*P*	**N**

2 erreurs en tout, 2/1000 erreur/itération

BR1 prédiction	*n*	*p*	*n*	*N*	*n*	*N*	*n*	*N*	*n*	*N*	...	*n*	*N*
BR1 comportement	**P**	**N**	*N*	*N*	**P**	*N*	*N*	*N*	**P**	*N*	...	*N*	*N*

0.25 erreur/itération (+1/1000 erreur/itération)

BR1 prédiction	*n*	*p*	*n*	*p*	*n*	*p*	*n*	*p*	*n*	*p*	...	*n*	*p*
BR1 comportement	**P**	**N**	**P**	**N**	**P**	**N**	**P**	**N**	**P**	**N**	...	**P**	**N**

1 erreur/itération

1.4 On suppose maintenant que l’on utilise un prédicteur avec historique avec 2 bits d’historique (prise en compte des deux derniers branchements exécutés) et 2 bits de prédiction comme ci dessus. Il y aura ainsi un prédicteur différent pour chacun des comportements des deux banchements précédents. Comme BR0 est (presque) toujours pris, en pratique, pour BR1, il y aura un prédicteur différent suivant que BR2 (à l’étape $i-1$) a été pris ou non pris. De même pour BR2, il y aura deux prédicteurs suivant que BR1 (à l’étape i) a été pris ou non pris.

On suppose tous les prédicteurs initialisés à “fortement non pris” et tout l’historique des branchements intialisé à “non pris”.

Dans ces conditions, déterminer la précision de la prédiction pour BR1 et BR2.

Rep : *BR1 :*

Comportement BR1		*P*	*N*	*N*	*N*	*P*	*N*	*N*	*N*	...	*P*	*N*	*N*	*N*
Comportement BR2		*P*	*N*	*P*	*N*	*P*	*N*	*P*	*N*	...	*P*	*N*	*P*	*N*
Historique (BR2/i-1)	*N*	*N*	*P*	*N*	*P*	*N*	*P*	*N*	*P*	...	*N*	*P*	*N*	*P*
Préd. BR2/i-1 NP	*N*	**N**	**N**	*N*	**N**	*N*	**N**	*N*	**N**	...	**N**	*N*	**N**	*N*
Préd. BR2/i-1 P	*N*	*N*	*N*	**N**	*N*	**n**	*n*	**N**	*N*	...	*N*	**n**	*n*	**N**
Qualité		*0*	*1*	*1*	*1*	*0*	*1*	*1*	*1*	...	*0*	*1*	*1*	*1*

0.25 errreur/itération

BR2 :

Comportement BR2		*P*	*N*	*P*	*N*	*P*	*N*	*P*	*N*	...	*P*	*N*	*P*	*N*
Comportement BR1		*P*	*N*	*N*	*N*	*P*	*N*	*N*	*N*	...	*P*	*N*	*N*	*N*
Historique (BR1/i)	*N*	*P*	*N*	*N*	*N*	*P*	*N*	*N*	*N*	...	*P*	*N*	*N*	*N*
Préd. BR1/i NP	*N*	*n*	**N**	**n**	**N**	*N*	**N**	**n**	**N**	...	*N*	**N**	**n**	**N**
Préd. BR1/i P	*N*	**n**	*n*	*n*	*n*	**p**	*p*	*p*	*p*	...	**P**	*P*	*P*	*P*
Qualité		*0*	*1*	*0*	*1*	*1*	*1*	*0*	*1*	...	*1*	*1*	*0*	*1*

0.25 erreur/itération (+1/1000 à cause des premières étapes où la prédiction quand BR1/i est pris est différente de p ou P. Après, on prédira correctement que quand i est mutiple de 4 (BR1 pris), il est aussi multiple de 2 (BR2 pris).)

2 Optimisation de boucles [8 pt]

On utilise le processeur superscalaire défini dans l'annexe 1

Soit la boucle suivante écrite en assembleur, qui travaille sur des vecteurs de `float`, `X[N], A[N], B[N], C[N]` et `D[N]` ; `R1` contient la valeur 0001 0000H et `R2` la valeur N. Les adresses des vecteurs sont :

```
0001 0000H X[0]
0001 1000H A[0]
0001 2000H B[0]
0001 3000H C[0]
0001 4000H D[0]
```

Soit la boucle assembleur

```
Boucle : LF   F1, 1000H(R1)
         LF   F2, 2000H(R1)
         LF   F3, 3000H(R1)
         LF   F4, 4000H(R1)
         FMUL F1, F1, F2
         FMUL F3, F3, F4
         FADD F1, F1, F3
         SF   F1, 0000H(R1)
         ADDI R1, R1, 4
         ADDI R2, R2, -1
         BNE  R2, Boucle
```

2.1 Donner le programme C équivalent.

Rep :
```
#define N 128 // ou toute autre valeur < 4k
float X[N], A[N],B[N],C[N],D[N];
for(int i=0; i<N; i++)
  x[i]=(a[i]*b[i])+(c[i]*d[i]);
```

2.2 On suppose d'abord que l'on utilise un processeur scalaire (une seule instruction démarrée par cycle). Optimisez la suite d'instructions. Quel est, en nombre de cycles, le temps d'exécution par itération de la boucle ?

Rep :
```
Boucle : LF F1, 11000H(R1)
         LF F2, 2000H(R1)
         LF F3, 3000H(R1)
         LF F4, 4000H(R1)
         FMUL F1, F1, F2
         FMUL F3, F3, F4
         (NOP X 2)
         FADD F1, F1, F3
         (NOP X 2)
         SF F1, 0000(R1)
         ADDI R1, R1, 4
         ADDI R2, R2, -1
         BNE R2, Boucle
```

13 cycles/itération

2.3 On passe maintenant à une version superscalaire du processeur. Donner l'exécution cycle par cycle de la boucle en plaçant les instructions dans les différents pipelines E0, E1, FA et FM. Quel est, en nombre de cycles, le temps d'exécution par itération de la boucle ?

cycles	E0	E1	FA	FM
1				
2				
...				

Rep :

cycles	*E0*	*E1*	*FA*	*FM*
1	*LF F1 ...*	*LF F2 ...*		
2	*LF F3 ...*	*LF F4 ...*		
3				*FMUL F1...*
4				*FMUL F3...*
5				
6				
7			*FADD F1...*	
8				
9	*ADDI R1, ...*	*ADDI R2, ...*		
10	*SF F1...*	*BNE R2...*		

2.4 Refaire la question précédente avec un déroulage de boucle d'ordre 4.

Rep :

```
Boucle : LF F1, 1000H(R1)
         LF F2, 2000H(R1)
         LF F3, 3000H(R1)
         LF F4, 4000H(R1)
         LF F5, 1004H(R1)
         LF F6, 2004H(R1)
         LF F7, 3004H(R1)
         LF F8, 4004H(R1)
         LF F9, 1008H(R1)
         LF F10, 2008H(R1)
         LF F11, 3008H(R1)
         LF F12, 4008H(R1)
         LF F13, 100CH(R1)
         LF F14, 200CH(R1)
         LF F15, 300CH(R1)
         LF F16, 400CH(R1)
         FMUL F1, F1, F2
         FMUL F3, F3, F4
         FMUL F5, F5, F6
         FMUL F7, F7, F8
         FMUL F9, F9, F10
         FMUL F11, F11, F12
         FMUL F13, F13, F14
         FMUL F15, F15, F16
         FADD F1, F1, F3
         FADD F5, F5, F7
         FADD F9, F9, F11
         FADD F13, F13, F15
         SF F1, 0000H(R1)
         SF F1, 0004H(R1)
         SF F1, 0008H(R1)
         SF F1, 000CH(R1)
         ADDI R1, R1, 16
         ADDI R2, R2, -4
         BNE R2, Boucle
```

Rep :

cycles	*E0*	*E1*	*FA*	*FM*
1	*LF F1 ...*	*LF F2 ...*		
2	*LF F3 ...*	*LF F4 ...*		
3	*LF F5...*	*LF F7...*		*FMUL F1...*
4	*LF F9...*	*LF F11...*		*FMUL F3...*
5	*LF F13...*	*LF F15...*		*FMUL F5...*
6				*FMUL F7...*
7			*FADD F1...*	*FMUL F9...*
8				*FMUL F11...*
9	*ADDI R1...*	*ADDI R2...*	*FADD F5...*	*FMUL F13...*
10	*SF F1...*			*FMUL F15...*
11			*FADD F9...*	
12	*SF F5...*			
13			*FADD F13...*	
14	*SF F9...*			
15				
16	*SF F13...*	*BNE R2...*		

16 cycles/4 itérations, soit 4 cycles par itération
NB : On a passé les `ADDI` *(dont celui de R1) avant les* `SF` *pour optimiser... Il faut donc ajuster les adresses des* `SF` *pour prendre en compte l'incrémentation de R1 effectuée avant.*

3 Caches [3pt]

Un processeur utilise un cache de données de 64 Ko, avec des lignes de 64 octets, à correspondance directe. Le cache utilise la réécriture avec écriture allouée (il y a des défauts de cache en écriture). Le processeur a des adresses sur 32 bits.

On considère l'extrait de programme C suivant, pour lequel les tableaux `X` et `Y` sont rangés successivement en mémoire à partir de l'adresse 1000 0000H (adresse de `X[0]`.)

```
float X[2*N], Y[N];
for (i=0 ; i<N ; i++)
  Y[i] = X[i+N] - X[i] ;
```

3.1 Quelles est le nombre de bits des champs étiquette et index de ce cache ?

Rep : 6 bits déplacement (adresse dans la ligne de 64o), 10 bits d'index (64ko cache/64o par ligne=1K lignes) et 16 bits d'étiquette (32-10-6).

3.2 Quel est le nombre de défaut de cache par itération quand N = 512 ?

Rep : @X[0] = 0x1000 0000=0001 0000 0000 0000 |0000 0000 00|00 0000→ligne 0
@X[512] = @X[0]+2k=0x1000 0800=0001 0000 0000 0000|0000 1000 00|00 0000→ligne 32
@Y[0] = 0x1000 1000=0001 0000 0000 0000 |0001 0000 00|00 0000→ligne 64
Les lignes de X[i], X[N+i] et Y[i] sont différentes. Pas de défauts dus à des conflits. Il y a donc 3 défauts (obligatoires) toutes les 64/4=16 itérations. Soit 3/16 défauts par itération

3.3 Pour quelle valeur de N (puissance de 2) a-t-on 3 défauts par itération ?

Rep : Il y a 3 défauts par itération si les index des adresses de X[0], X[N] et Y[0] (qui est à l'adresse de X[2N]) sont les mêmes, soit @X[N] = @X[0] + 2^{16} soit N=2^{16-2}.
*Toutes les valeurs N=k*P, avec P=2^{14} et $k \in \{1, 2, ...\}$ provoquent 3 défauts par itération*

4 Mémoire virtuelle [5pt]

On suppose un processeur dont la taille des pages fait 1 ko. Les adresses virtuelles sont codées sur 64 bits, mais les adresses physiques ne sont que sur 40 bits (adressage maximum de 1 To).

4.1 Quelle est le nombre de bits nécessaire pour coder l'adresse d'une page ? Quelle serait la taille en octets de la table des pages de chaque process, en supposant qu'une seule table est utilisée ?

Rep : *Adresse/page=10 bits. Adresse de page = 40-10=30 bits. Dans une une page de 1ko, on peut stocker 256×4o, soit 256@/page. Comme il faut stocker 2^{30} pages, la table des pages fait 2^{30-8} octets, soit 4Mo.*

4.2 Pour réduire l'occupation de la mémoire, on conçoit une table à plusieurs niveaux. L'adresse virtuelle est découpée en champs permettant d'accéder à un des niveaux de la table. Dans chaque niveau, on fait en sorte de n'accèder qu'à une seule page. Combien peut-on stocker d'adresses de page dans une page ? Indiquer comment on doit découper l'adresse de page pour réaliser cet accès multiniveaux ? Quel est le nombre de niveaux de la table ?

Rep : *On met 256 adresses dans une page de 1ko. On découpe l'adresse de page en champs de 8 bits. Soit C1 C2 C3 C4 ces champs et PT1 l'adresse de la première table des pages (stockée dans la processeur). Le contenu de la mémoire en PT1+C1*4 donne l'adresse de la page de la 2ème table PT2, PT2*2^{10} est l'adresse de la table et en ajoutant C2*4, on peut lire l'adresse de la page de la 3ème table PT3. On procède de même pour PT3 et C3 qui donnent l'adresse de la page de la dernière table PT4. L'adresse de la page cherchée est en PT4*2^{10}+C4*4. Il faut donc parcourir 4 tables pour avoir les 30 bits.*

4.3 Soit le programme C suivant :

```
constant int N = 1000000000;
float A[N],sum;
...
for (int i=0; i<N; i++)
  sum += A[i];
```

A l'itération 0, il y a un défaut de TLB sur `A[0]`. Le processeur va alors parcourir les différents niveaux de la table des pages, stocker les lignes accédées dans le cache (et nous supposerons qu'elle y restent le temps de l'exercice), et mettre à jour l'entrée de TLB (qui restera également).

Pour quelle valeur de `i` y aura-t-il à nouveau un défaut de TLB ?

Rep : *Quand @A[i]=@A[0]+1k soit i*4=1k, soit i=256*

En supposant que les lignes de cache font 32o, pour quelle valeur de `i` le défaut de TLB provoquera-t-il aussi un défaut de cache ?

Rep : *Une adresse de page fait 32 bits (30 précisément), soit 4o. Dans une ligne, on stocke 32/4 adresses. Dans la ligne de cache, il y a donc les 8 premières adresses de pages. Au bout de 8 pages, les adresses de pages ne seront plus dans le cache. Cela arrive pour i*4=8k et i=2^{11}=2048.*

ANNEXE 1

Soit un processeur superscalaire à ordonnancement statique qui a les caractéristiques suivantes :

- les instructions sont de longueur fixe (32 bits)
- Il a 32 registres entiers (dont R0=0) de 32 bits et 32 registres flottants (de F0 à F31) de 32 bits.
- Il peut lire et exécuter 4 instructions par cycle.
- L'unité entière contient deux pipelines d'exécution entière sur 32 bits, soit deux additionneurs, deux décaleurs. Tous les court-circuits possibles sont implantés.
- L'unité flottante contient un pipeline flottant pour l'addition et un pipeline flottant pour la multiplication.
- L'unité Load/Store peut exécuter jusqu'à deux chargements par cycle, mais ne peut effectuer qu'un *load* et un `store` simultanément. Elle ne peut effectuer qu'un seul `store` par cycle.
- Le processeur dispose d'un mécanisme de prédiction de branchement qui permet de *brancher* en 1 cycle si la prédiction est correcte. Les sauts et branchements ne sont pas retardés.

La Table 1 donne les instructions disponibles et le pipeline qu'elles utilisent : E0 et E1 sont les deux pipelines entiers, FA est le pipeline flottant de l'addition et FM le pipeline flottant de la multiplication. Les instructions peuvent être exécutées simultanément si elles utilisent chacune un pipeline séparé. L'addition et la multiplication flottante sont pipelinées. La division flottante n'est pas pipelinée (une division ne peut commencer que lorsque la division précédente est terminée). L'ordonnancement est statique.

JEU D'INSTRUCTIONS (extrait)

Code	Instruction	Latence	Pipeline	Signification
LF	LF Fi, dép.(Ra)	2	E0 ou E1	Fi ← M (Ra + dépl.16 bits signé)
SF	SF Fi, dép.(Ra)	0	E0	Fi → M (Ra + dépl.16 bits signé)
ADD	ADD Rd,Ra, Rb	1	E0 ou E1	Rd ← Ra + Rb
ADDI	ADDI Rd, Ra, IMM	1	E0 ou E1	Rd ← Ra + IMM (16 bits signé)
SUB	SUB Rd,Ra, Rb	1	E0 ou E1	Rd ← Ra - Rb
FADD	FADD Fd, Fa, Fb	3	FA	Fd ← Fa + Fb
FMUL	FMUL Fd, Fa, Fb	3	FM	Fd ← Fa × Fb
BEQ	BEQ Ri, dépl	1	E1	si Ri=0 alors CP ← NCP + depl
BNE	BNE Ri, dépl	1	E1	si Ri≠ 0 alors CP ← NCP + depl